01 — Python Essentials

Goal: write small correct programs quickly, without thinking about syntax. Everything here appears in **Q1** and is the base for all other questions.

1. Variables and types

```
n = 42            # int
price = 19.99     # float
name = "Ama"      # str
ready = True      # bool

# Conversion — you will need these constantly
int("42")     # 42
str(42)       # "42"
float("3.5")  # 3.5
int(7 / 2)    # 3   (7 / 2 is 3.5; int() cuts the decimal)
7 // 2        # 3   (integer division — PREFER this)
7 % 2         # 1   (remainder — used for even/odd, digits, cycles)
2 ** 10       # 1024 (power)
abs(-5)       # 5
```

`%` and `//` are exam gold. Examples:

- last digit of `n` → `n % 10`
- remove last digit → `n // 10`
- is even → `n % 2 == 0`
- wrap around a list of size `k` → `i % k`

2. Conditions

```python
def category(age):
    if age < 13:
        return "child"
    elif age < 20:
        return "teen"
    else:
        return "adult"

# Chained comparison (very Pythonic):
if 0 <= x < 100: ...

# Combine:
if a > 0 and b > 0: ...
if x < 0 or x > 9: ...
if not found: ...
```

3. Loops

```python
for i in range(5):        # 0 1 2 3 4
for i in range(2, 5):     # 2 3 4
for i in range(10, 0, -1):# 10 9 ... 1  (counting down)

nums = [3, 1, 4]
for x in nums:            # loop over values
for i, x in enumerate(nums):   # index AND value: (0,3) (1,4) ...
for i in range(len(nums)):     # when you need to modify nums[i]

# while — when you don't know how many steps
while n > 0:
    n //= 10

break     # exit the loop now
continue  # skip to next iteration
```

4. Functions

CodeSignal gives you a function to fill in — you `return` the answer (you do NOT print it):

```python
def solution(a, b):
    return a + b
```

Common beginner mistake: using `print` instead of `return`. The grader sees `None` and marks you wrong.

5. Digits of a number — classic Q1 pattern

Two tools let you take a number apart, one digit at a time, from the right:

- `n % 10` → the **last digit** (`1234 % 10` = 4)
- `n // 10` → the number with its **last digit removed** (`1234 // 10` = 123)

Loop those two until n reaches 0 and you've visited every digit:

```
def digit_sum(n):
    total = 0
    while n > 0:
        total += n % 10   # take last digit
        n //= 10          # drop last digit
    return total
```

Trace `digit_sum(123)`: grab 3 (total 3, n→12), grab 2 (total 5, n→1), grab 1 (total 6, n→0), stop. Answer 6. This **peel-the-digits** loop powers reverse-a-number, count-digits, and palindrome-number problems.

```
# Or the string trick (also fine, often simpler!):
def digit_sum(n):
    return sum(int(d) for d in str(n))
```

Here `str(n)` turns 123 into `"123"`, then `int(d)` turns each character back into a number. Turning a number into a string to walk its digits is completely valid on the exam.

6. min / max / sum — know these cold

```
nums = [4, 7, 1]
max(nums)   # 7
min(nums)   # 1
sum(nums)   # 12
len(nums)   # 3
sorted(nums)            # [1, 4, 7] (new list)
sorted(nums, reverse=True)  # [7, 4, 1]
max(a, b)   # works on two values too
```

7. Tracking a "best so far" — the most common loop shape in the whole exam

```
def largest_gap(nums):
    best = 0                                # the biggest gap found s
    for i in range(1, len(nums)):           # start at 1 so nums[i-1]
        gap = abs(nums[i] - nums[i-1])      # distance between this i
        if gap > best:                      # found a bigger one?
            best = gap                      # remember it
    return best
```

Memorize the shape: **start `best` at a safe value → loop → compare → update.** It's the same "running variable" idea as summing or counting, just keeping the *best* instead of a total. (Note `range(1, len(nums))`: we begin at 1 because the loop looks back at `nums[i-1]`, which would be off the front of the list if `i` were 0.)

Exercises (do them in `practice/q1_warmups.py`)

1. `century(year)` — which century is a year in? (year 1905 → 20, year 1700 → 17)
2. `sum_odd(n)` — sum of all odd numbers from 1 to n.
3. `reverse_number(n)` — 1234 → 4321 (try BOTH the math way and the string way).
4. `is_leap(year)` — divisible by 4, except centuries unless divisible by 400.
5. `count_digits_divisible(n)` — how many digits of n divide n evenly? (watch out for digit 0!)